

Relatório Lista 4: Sistema de autocomple com Trie

Grupo: Angel Jhesua Machado Rodriguez e João Guilherme de Souza Gomes **Professor:** Matheus Werner
Data: Junho de 2026

1 Representação da Trie e definições iniciais

As Tries podem ser retratadas abstratamente de duas maneiras principais. Na primeira, os caracteres das palavras ficam nas arestas e nos nós está o indicador de fim de palavra. Na segunda, os caracteres são guardados diretamente pelos nós, juntamente com os seus outros atributos. A representação da estrutura no código depende de qual conceito é adotado. No presente trabalho, o primeiro modelo foi o escolhido, uma vez que se optou por priorizar a velocidade de execução da busca. Nessa abordagem, o caractere não faz parte explicitamente da estrutura interna do nó, em vez disso, adota-se um alfabeto fixo onde cada nó armazena um array cujas posições são indexadas pelos próprios caracteres do alfabeto.

Definiu-se um “alfabeto” fixo de 36 letras, formado pelas 26 letras do alfabeto latino e os números de 0 a 9. Essa restrição tornou possível que os casos de normalização de chaves ou transformação de caracteres em índices pudessem ser feitos inteiramente através da manipulação da tabela ASCII.

O sistema é *case-insensitive*, para garantir esse comportamento o método `toSearchKey` atua como um normalizador. Antes de qualquer inserção ou busca, as letras da *string* são totalmente convertidas para letras maiúsculas e têm todos os seus caracteres de espaço removidos. Ele se utiliza de outros dois métodos, o primeiro, `removeSpaces`, remove os espaços (através do método `erase` para strings), enquanto o segundo, `toUpperCase`, se aproveita da tabela ASCII que possui a propriedade de os caracteres serem representados por valores inteiros sequenciais, logo, a distância entre qualquer letra minúscula e sua respectiva maiúscula é uma constante equivalente a 32. Ao subtrair o deslocamento calculado por `'a' - 'A'`, o valor binário do caractere é alterado para a sua versão em maiúscula.

O mapeamento de caracteres para os índices do vetor de filhos (`children`) foi implementado no método `charToIndex`. Através da manipulação da tabela ASCII, os dígitos de '0' a '9' são mapeados para as posições de 0 a 9, enquanto as letras de 'A' a 'Z' são deslocadas para ocupar as posições de 10 a 35.

2 Funcionamento dos Métodos `insert`, `contains` e `autocomplete`

A manipulação da árvore Trie ocorre através de três operações principais: inserção de novos elementos, verificação de existência (busca exata) e a recuperação de sugestões baseada em prefixos (autocomplete).

Inserção (`insert`) e Busca Exata (`contains`)

O método de inserção (`insert`) é responsável por preencher a árvore. Ao receber um ponteiro para um objeto `Game`, o algoritmo extrai seu título, o normaliza e inicia uma travessia a partir da raiz (`root`). Para cada caractere da chave, calcula-se o índice correspondente no alfabeto. A travessia ocorre descendo os níveis da árvore; caso o ponteiro no índice correspondente ao caractere atual seja nulo (`nullptr`), um novo `TrieNode` é dinamicamente alocado. Ao atingir o final da *string*, o último nó recebe o ponteiro do objeto `Game` e a flag `isEndOfTitle` é marcada como `true`, construindo o caminho como um título válido no catálogo.

Analogamente, o método `contains` verifica a existência de um jogo específico. Ele percorre a árvore seguindo os índices da chave buscada. Se, em qualquer ponto da descida, o caminho apontar para um nó nulo, o método é interrompido, retornando `false` (indicando que a sequência não existe na Trie). Caso o loop termine e o último nó seja alcançado com sucesso, o método não retorna simplesmente `true`, mas sim o valor da variável `isEndOfTitle` daquele nó. Isso é importante para diferenciar títulos reais de caminhos que são apenas prefixos de palavras maiores (por exemplo, garantir que buscar por “HALF” retorne `false` se apenas “HALF LIFE” estiver cadastrado).

Busca por Prefixo (`autocomplete`)

A operação de autocomplete é dividida em duas fases principais. A primeira utiliza o método auxiliar interno `takeNode`. Ele serve exclusivamente para localizar e retornar o nó que representa o último caractere

do prefixo digitado pelo usuário. Se o prefixo não existir na árvore, o método retorna nulo, encerrando o autocomplete com um vetor vazio. Uma vez que o nó correspondente a última letra do prefixo é alcançado, ocorre a segunda fase: a coleta de todas as palavras válidas utilizando uma Busca em Profundidade.

Para isso, utiliza-se um vetor (`std::vector<TrieNode*>`) como uma estrutura de Pilha (*Stack*). O nó do prefixo é empilhado e, enquanto a pilha não estiver vazia, o algoritmo desempilha o nó do topo, verifica se ele representa o fim de um título (adicionando seu jogo ao vetor de resultados caso seja) e empilha todos os seus filhos não nulos. Por fim, o vetor com todos os jogos encontrados é ordenado e os k primeiros são retornados.

3 Análise de Custo Computacional

Denotemos por L , o tamanho do título do jogo após a conversão para chave de busca; p , o tamanho do prefixo buscado após a conversão; s , o número de nós visitados na subárvore delimitada pelo prefixo; e r , o número total de jogos encontrados dentro dessa mesma subárvore.

- **Inserção (insert) - $\mathcal{O}(L)$:** O custo computacional da inserção é linear em relação ao tamanho da chave de busca. O algoritmo executa um laço que itera exatamente L vezes, processando um caractere por iteração. Em cada passo, a conversão do caractere para índice, a verificação da existência do nó e a eventual alocação de memória ocorrem em tempo constante $\mathcal{O}(1)$ devido ao uso do *array* de filhos estático. Logo, o custo total depende somente de L .
- **Busca Exata (contains) - $\mathcal{O}(L)$:** Análogamente, a busca exata depende unicamente do comprimento da palavra procurada. No pior caso (quando a palavra existe ou quando a divergência ocorre apenas no último caractere), o algoritmo desce L níveis na árvore. Como o acesso a cada transição de caractere é feito através de indexação direta em $\mathcal{O}(1)$, a complexidade final é dada por $\mathcal{O}(L)$.
- **Busca por Prefixo (autocomplete) - $\mathcal{O}(p+s+r \log r)$:** A complexidade do autocomplete é composta pela soma dos custos de suas três etapas operacionais distintas:
 1. **Navegação inicial:** O método `takeNode` consome $\mathcal{O}(p)$ operações para descer a árvore até o nó correspondente ao final do prefixo buscado.
 2. **Travessia da subárvore:** A partir do nó do prefixo, a Busca em Profundidade visita todos os nós descendentes válidos. O tempo de execução desta etapa é diretamente proporcional ao número de nós na subárvore, resultando em um custo de $\mathcal{O}(s)$.
 3. **Ordenação dos resultados:** Durante a DFS, r jogos são identificados e armazenados no vetor de resultados. Antes da extração dos k elementos solicitados, este vetor de tamanho r é ordenado utilizando o algoritmo *Merge Sort*, o qual tem um custo de $\mathcal{O}(r \log r)$. A seleção final de k elementos tem um custo linear $\mathcal{O}(k)$ que acaba sendo absorvido pela complexidade dominante da ordenação.

4 Comparação e Uso de Memória

A implementação ingênua do autocomplete poderia consistir em colocar todos os jogos num grande array linear. A cada busca, seria necessário a lista inteira, checando se o título de cada jogo começa com o prefixo digitado. Se a nossa base tiver N jogos cadastrados, essa busca custaria aproximadamente $\mathcal{O}(N \times p)$. Para um catálogo muito pequeno, isso funciona, mas se o sistema escalar para milhares ou milhões de jogos, o autocomplete seria inviável.

Com a Trie, a vantagem é que eliminamos o fator N da etapa de busca. O tempo necessário para encontrar o início do prefixo depende estritamente do tamanho do próprio prefixo (p). Isso faz com que a resposta seja muito mais rápida.

Toda essa velocidade da Trie exige um maior consumo de memória. Enquanto um vetor simples ocupa apenas o espaço exato para guardar os ponteiros dos jogos de forma sequencial, a Trie cria um conjunto de nós interconectados na memória heap. O tamanho do alfabeto escolhido determina o tamanho do vetor que cada nó da árvore carrega. Em sistemas de 64 bits, cada ponteiro consome 8 *bytes*. Isso significa que cada nó da nossa árvore ocuparia pelo menos 288 *bytes* de memória, mesmo que a maioria desses ponteiros esteja apontando para o vazio (`nullptr`).

Apesar de cada nó individual ser "caro" para a memória, a Trie se torna viável na prática graças a sua estrutura. Títulos que comecem com as mesmas letras, compartilham exatamente os mesmos caminhos e nós iniciais na árvore para a sequência de chaves. Ou seja, a Trie reaproveita os nós que já existem. Quanto maior e mais denso for o volume de dados inserido, maior será essa taxa de reaproveitamento.

5 Comparação e Uso de Memória

Em uma **solução ingênua** baseada unicamente em vetor, o autocomplete exigiria uma iteração por todo o catálogo de jogos, comparando o prefixo desejado com cada título. Isso resultaria em uma complexidade de tempo linear em relação ao tamanho total do catálogo, tornando a busca inviável em bases de dados muito densas.

Com a **Trie**, a complexidade temporal se desvincula da quantidade total de jogos, dependendo apenas do tamanho do prefixo procurado e da profundidade da sua subárvore específica. O tempo de resposta é significativamente reduzido, viabilizando buscas em tempo real.

O revés natural da Trie é o **consumo de memória**. Cada `TrieNode` carrega um *array* de 36 ponteiros (neste escopo de `ALPHABET_SIZE = 36`). Em uma arquitetura de 64 bits, apenas os ponteiros de um único nó ocupam 288 *bytes*, mesmo que muitos permaneçam nulos (`nullptr`).

Contudo, este custo é drasticamente mitigado pelo **compartilhamento de prefixos**. Jogos que possuem inícios de título idênticos (como “Half Life” e “Halo”, que compartilham a sequência de chaves correspondente a “HA-L”) reaproveitam os mesmos nós no caminho inicial da árvore. Essa sobreposição natural em catálogos textuais impede a duplicação desnecessária de caracteres na memória, justificando o balanço positivo entre o gasto de armazenamento e o imenso ganho de performance operacional.